

Data Analysis and Plotting in Python with Pandas

Adel Dabah, Jülich Supercomputing Centre, Forschungszentrum Jülich, 30 March 2026

MATSE Version



My Motivation

- I like Python
- I like plotting data
- I like sharing
- I think Pandas is awesome and you should use it too
- ...*but I'm no Python expert!*

*Motto: »**Pandas as early as possible!**«*



My Motivation

- I like Python
- I like plotting data
- I like sharing
- I think Pandas is awesome and you should use it too
- *...but I'm no Python expert!*

*Motto: »**Pandas as early as possible!**«*

Task Outline

- Task 1
- Task 2
- Task 3
- Task 4
- Task 5
- Task 6
- Task 7
- Task 7B
- Task 8
- Task 8B



Course Setup

- Originally, a 3 h course (including break)
- This is the two-day MATSE lecture, with plenty of space and some new material
- Alternating between lecture and hands-on



Course Setup

- Originally, a 3 h course (including break)
- This is the two-day MATSE lecture, with plenty of space and some new material
- Alternating between lecture and hands-on
- Please now open Jupyter Notebook of this session from Sciebo



About Pandas



- Python package (Python 3)
- For data analysis and manipulation
- With data structures (multi-dimensional table; time series), operations
- Name from »**Panel Data**« (multi-dimensional time series in economics)
- Since 2008
- <https://pandas.pydata.org/>
- Install via PyPI: `pip install pandas`
- Cheatsheet: https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

Pandas Cohabitation

- Pandas works great together with other established Python tools
 - [Jupyter Notebooks](#)
 - Plotting with `matplotlib`
 - Numerical analysis with `numpy`
 - Modelling with `statsmodels`, `scikit-learn`
 - Nicer plots with `seaborn`, `altair`, `plotly`
 - Performance enhancement with [Cython](#), [Numba](#), ...
- Tools building up on Pandas: [cuDF](#) (GPU-accelerated DataFrames in [Rapids](#)), [pyarrow](#) (Apache Arrow bindings in Python) ...

First Steps



First Steps

In [1]: `import pandas`



First Steps

```
In [1]: import pandas
```

```
In [2]: import pandas as pd
```

```
In [3]: pd.__version__
```

```
Out[3]: '2.2.3'
```



First Steps

```
In [1]: import pandas
```

```
In [2]: import pandas as pd
```

```
In [3]: pd.__version__
```

```
Out[3]: '2.2.3'
```

```
In [4]: %pdoc pd
```

Class docstring:

pandas - a powerful data analysis and manipulation library for Python

=====

****pandas**** is a Python package providing fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, ****real world**** data analysis in Python. Additionally, it has the broader goal of becoming ****the most powerful and flexible open source data analysis / manipulation tool available in any language****. It is already well on its way toward this goal.

Main Features

Here are just a few of the things that pandas does well:

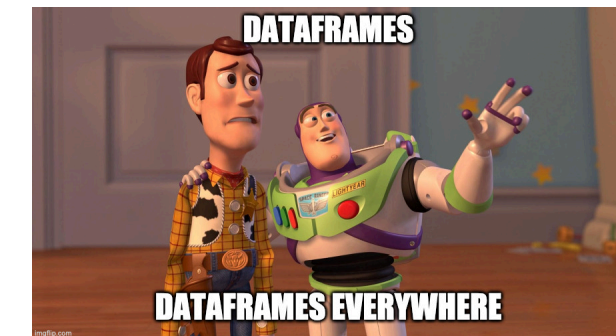
- Easy handling of missing data in floating point as well as non-floating point data.



DataFrames

It's all about DataFrames

- Data containers of Pandas:
 - Linear: `Series`
 - Multi Dimension: `DataFrame`
- `Series` is *only* special (1D) case of `DataFrame`
- → We use `DataFrame`s as the more general case here



DataFrames

Construction

- To show features of `DataFrame` , let's construct one and show by example!
- Many construction possibilities
 - From lists, dictionaries, `numpy` objects
 - From CSV, HDF5, JSON, Excel, HTML, fixed-width files
 - From pickled Pandas data
 - From clipboard
 - *From Feather, Parquet, SAS, SQL, Google BigQuery, STATA*

DataFrames

Examples, finally



DataFrames

Examples, finally

```
In [5]: ages = [41, 56, 56, 57, 39, 59, 43, 56, 38, 60]
```



DataFrames

Examples, finally

```
In [5]: ages = [41, 56, 56, 57, 39, 59, 43, 56, 38, 60]
```

```
In [6]: pd.DataFrame(ages)
```

```
Out[6]:
```

	0
0	41
1	56
2	56
3	57
4	39
5	59
6	43
7	56
8	38
9	60



DataFrames

Examples, finally

```
In [5]: ages = [41, 56, 56, 57, 39, 59, 43, 56, 38, 60]
```

```
In [6]: pd.DataFrame(ages)
```

```
Out[6]:
```

	0
0	41
1	56
2	56
3	57
4	39
5	59
6	43
7	56
8	38
9	60

```
In [7]: df_ages = pd.DataFrame(ages)
df_ages.head(3)
```

```
Out[7]:
```

	0
0	41
1	56
2	56



Task 1

- Create data frame with
 - 6 names of dinosaurs,
 - their favourite prime number,
 - and their favorite color.
- Play around with the frame
- Tell me when you're done with status icon in BigBlueButton: 👍



Task 1

- Create data frame with
 - 6 names of dinosaurs,
 - their favourite prime number,
 - and their favorite color.
- Play around with the frame
- Tell me when you're done with status icon in BigBlueButton: 👍

```
In [26]: happy_dinos = {  
          "Dinosaur Name": [],  
          "Favourite Prime": [],  
          "Favourite Color": []  
        }  
#df_dinos =
```



Task 1

- Create data frame with
 - 6 names of dinosaurs,
 - their favourite prime number,
 - and their favorite color.
- Play around with the frame
- Tell me when you're done with status icon in BigBlueButton: 👍

```
In [26]: happy_dinos = {  
    "Dinosaur Name": [],  
    "Favourite Prime": [],  
    "Favourite Color": []  
}  
#df_dinos =
```

```
In [27]: happy_dinos = {  
    "Dinosaur Name": ["Aegyptosaurus", "Tyrannosaurus", "Panoplosaurus", "Isisaurus", "Triceratops", "Velociraptor"],  
    "Favourite Prime": ["4", "8", "15", "16", "23", "42"],  
    "Favourite Color": ["blue", "white", "blue", "purple", "violet", "gray"]  
}  
df_dinos = pd.DataFrame(happy_dinos).set_index("Dinosaur Name")  
df_dinos.T
```

```
Out[27]:
```

Dinosaur Name	Aegyptosaurus	Tyrannosaurus	Panoplosaurus	Isisaurus	Triceratops	Velociraptor
Favourite Prime	4	8	15	16	23	42
Favourite Color	blue	white	blue	purple	violet	gray



Reading External Data

(Links to documentation)

- `.read_json()`
- `.read_csv()`
- `.read_hdf5()`
- `.read_excel()`

Example:

```
{  
  "Character": ["Sawyer", "...", "Walt"],  
  "Actor": ["Josh Holloway", "...", "Malcolm David Kelley"],  
  "Main Cast": [true, "...", false]  
}
```



Reading External Data

(Links to documentation)

- `.read_json()`
- `.read_csv()`
- `.read_hdf5()`
- `.read_excel()`

Example:

```
{
  "Character": ["Sawyer", "...", "Walt"],
  "Actor": ["Josh Holloway", "...", "Malcolm David Kelley"],
  "Main Cast": [true, "...", false]
}
```

```
In [33]: pd.read_json("data-lost.json").set_index("Character").sort_index()
```

Out[33]:

	Actor	Main Cast
Character		
Hurley	Jorge Garcia	True
Jack	Matthew Fox	True
Kate	Evangeline Lilly	True
Locke	Terry O'Quinn	True
Sawyer	Josh Holloway	True
Walt	Malcolm David Kelley	False



Task 2

- Read in `data-nest.csv` to `DataFrame`; call it `df`
(Data was produced with [JUBE](#))
- Get to know it and play a bit with it
- Tell me when you're done with status icon in BigBlueButton: 👍

In [34]: `!head data-nest.csv`

```
id,Nodes,Tasks/Node,Threads/Task,Runtime Program / s,Scale,Plastic,Avg. Neuron B
uild Time / s,Min. Edge Build Time / s,Max. Edge Build Time / s,Min. Init. Time
/ s,Max. Init. Time / s,Presim. Time / s,Sim. Time / s,Virt. Memory (Sum) / kB,L
ocal Spike Counter (Sum),Average Rate (Sum),Number of Neurons,Number of Connecti
ons,Min. Delay,Max. Delay
5,1,2,4,420.42,10,true,0.29,88.12,88.18,1.14,1.20,17.26,311.52,46560664.00,82549
9,7.48,112500,1265738500,1.5,1.5
5,1,4,4,200.84,10,true,0.15,46.03,46.34,0.70,1.01,7.87,142.97,46903088.00,80286
5,7.03,112500,1265738500,1.5,1.5
5,1,2,8,202.15,10,true,0.28,47.98,48.48,0.70,1.20,7.95,142.81,47699384.00,80286
5,7.03,112500,1265738500,1.5,1.5
5,1,4,8,89.57,10,true,0.15,20.41,23.21,0.23,3.04,3.19,60.31,46813040.00,821491,
7.23,112500,1265738500,1.5,1.5
5,2,2,4,164.16,10,true,0.20,40.03,41.09,0.52,1.58,6.08,114.88,46937216.00,80286
5,7.03,112500,1265738500,1.5,1.5
5,2,4,4,77.68,10,true,0.13,20.93,21.22,0.16,0.46,3.12,52.05,47362064.00,821491,
7.23,112500,1265738500,1.5,1.5
5,2,2,8,79.60,10,true,0.20,21.63,21.91,0.19,0.47,2.98,53.12,46847168.00,821491,
7.23,112500,1265738500,1.5,1.5
5,2,4,8,37.20,10,true,0.13,10.08,11.60,0.10,1.63,1.24,23.29,47065232.00,818198,
7.33,112500,1265738500,1.5,1.5
5,3,2,4,96.51,10,true,0.15,26.54,27.41,0.36,1.22,3.33,64.28,52256880.00,813743,
7.27,112500,1265738500,1.5,1.5
```



Task 2

- Read in `data-nest.csv` to `DataFrame`; call it `df`
(Data was produced with [JUBE](#))
- Get to know it and play a bit with it
- Tell me when you're done with status icon in BigBlueButton: 👍

In [34]: `!head data-nest.csv`

```
id,Nodes,Tasks/Node,Threads/Task,Runtime Program / s,Scale,Plastic,Avg. Neuron B
uild Time / s,Min. Edge Build Time / s,Max. Edge Build Time / s,Min. Init. Time
/ s,Max. Init. Time / s,Presim. Time / s,Sim. Time / s,Virt. Memory (Sum) / kB,L
ocal Spike Counter (Sum),Average Rate (Sum),Number of Neurons,Number of Connecti
ons,Min. Delay,Max. Delay
5,1,2,4,420.42,10,true,0.29,88.12,88.18,1.14,1.20,17.26,311.52,46560664.00,82549
9,7.48,112500,1265738500,1.5,1.5
5,1,4,4,200.84,10,true,0.15,46.03,46.34,0.70,1.01,7.87,142.97,46903088.00,80286
5,7.03,112500,1265738500,1.5,1.5
5,1,2,8,202.15,10,true,0.28,47.98,48.48,0.70,1.20,7.95,142.81,47699384.00,80286
5,7.03,112500,1265738500,1.5,1.5
5,1,4,8,89.57,10,true,0.15,20.41,23.21,0.23,3.04,3.19,60.31,46813040.00,821491,
7.23,112500,1265738500,1.5,1.5
5,2,2,4,164.16,10,true,0.20,40.03,41.09,0.52,1.58,6.08,114.88,46937216.00,80286
5,7.03,112500,1265738500,1.5,1.5
5,2,4,4,77.68,10,true,0.13,20.93,21.22,0.16,0.46,3.12,52.05,47362064.00,821491,
7.23,112500,1265738500,1.5,1.5
5,2,2,8,79.60,10,true,0.20,21.63,21.91,0.19,0.47,2.98,53.12,46847168.00,821491,
7.23,112500,1265738500,1.5,1.5
5,2,4,8,37.20,10,true,0.13,10.08,11.60,0.10,1.63,1.24,23.29,47065232.00,818198,
7.33,112500,1265738500,1.5,1.5
5,3,2,4,96.51,10,true,0.15,26.54,27.41,0.36,1.22,3.33,64.28,52256880.00,813743,
7.27,112500,1265738500,1.5,1.5
```



Read CSV Options

- See also full [API documentation](#)
- Important parameters
 - `sep` : Set separator (for example `:` instead of `,`)
 - `header` : Specify info about headers for columns; able to use multi-index for columns!
 - `names` : Alternative to `header` – provide your own column titles
 - `usecols` : Don't read whole set of columns, but only these; works with any list (`range(0:20:2)`)...
 - `skiprows` : Don't read in these rows
 - `na_values` : What string(s) to recognize as `N/A` values (which will be ignored during operations on data frame)
 - `parse_dates` : Try to parse dates in CSV; different behaviours as to provided data structure; optionally used together with `date_parser`
 - `compression` : Treat input file as compressed file ("infer", "gzip", "zip", ...)
 - `decimal` : Decimal point divider – for German data...

```
pandas.read_csv(filepath_or_buffer, sep=<object object>, delimiter=None, header='infer', names=None,
index_col=None, usecols=None, squeeze=False, prefix=None, mangle_dupe_cols=True, dtype=None,
engine=None, converters=None, true_values=None, false_values=None, skipinitialspace=False,
skiprows=None, skipfooter=0, nrows=None, na_values=None, keep_default_na=True, na_filter=True,
verbose=False, skip_blank_lines=True, parse_dates=False, infer_datetime_format=False,
keep_date_col=False, date_parser=None, dayfirst=False, cache_dates=True, iterator=False, chunksize=None,
compression='infer', thousands=None, decimal='.', lineterminator=None, quotechar='"', quoting=0,
doublequote=True, escapechar=None, comment=None, encoding=None, dialect=None, error_bad_lines=True,
warn_bad_lines=True, delim_whitespace=False, low_memory=True, memory_map=False, float_precision=None,
storage_options=None)
```

Slicing of Data Frames

- Slicing: Select a sub-range / sub-set of entire data frame
- Pandas documentation: [Detailed documentation](#), [short documentation](#)



Slicing of Data Frames

- Slicing: Select a sub-range / sub-set of entire data frame
- Pandas documentation: [Detailed documentation](#), [short documentation](#)

Quick Slices

- Use square-bracket operators to slice data frame quickly: `[]`
 - Use column name to select column
 - Use numerical value to select row
- Example: Select only columnn `C` from `df_demo`

```
In [36]: df_demo.head(3)
```

Out[36]:

	A	B	C	D	E
0	1.2	2018-02-26	-2.718282	This	Same
1	1.2	2018-02-26	1.718282	column	Same
2	1.2	2018-02-26	-1.304068	has	Same



Slicing of Data Frames

- Slicing: Select a sub-range / sub-set of entire data frame
- Pandas documentation: [Detailed documentation](#), [short documentation](#)

Quick Slices

- Use square-bracket operators to slice data frame quickly: `[]`
 - Use column name to select column
 - Use numerical value to select row
- Example: Select only column `C` from `df_demo`

```
In [36]: df_demo.head(3)
```

```
Out[36]:
```

	A	B	C	D	E
0	1.2	2018-02-26	-2.718282	This	Same
1	1.2	2018-02-26	1.718282	column	Same
2	1.2	2018-02-26	-1.304068	has	Same

```
In [37]: df_demo['C']
```

```
Out[37]:
```

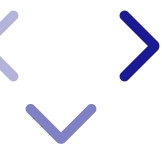
0	-2.718282
1	1.718282
2	-1.304068
3	0.986231
4	-0.718282

Name: C, dtype: float64



Adding to Existing Data Frame

- Add new columns with `frame["new col"] = something` or `.insert()`
- Add new rows with `frame.append()`
- Combine data frames
 - *Concat*: Combine several data frames along an axis
 - *Merge*: Combine data frames on basis of common columns; database-style
 - (Join)
 - See user guide [on merging](#)



Adding to Existing Data Frame

- Add new columns with `frame["new col"] = something` or `.insert()`
- Add new rows with `frame.append()`
- Combine data frames
 - *Concat*: Combine several data frames along an axis
 - *Merge*: Combine data frames on basis of common columns; database-style
 - (Join)
 - See user guide [on merging](#)

In [52]: `df_demo.head(3)`

Out[52]:

	A	B	C	D	E
0	1.2	2018-02-26	-2.718282	This	Same
1	1.2	2018-02-26	1.718282	column	Same
2	1.2	2018-02-26	-1.304068	has	Same



Adding to Existing Data Frame

- Add new columns with `frame["new col"] = something` or `.insert()`
- Add new rows with `frame.append()`
- Combine data frames
 - *Concat*: Combine several data frames along an axis
 - *Merge*: Combine data frames on basis of common columns; database-style
 - (Join)
 - See user guide [on merging](#)

```
In [52]: df_demo.head(3)
```

Out[52]:

	A	B	C	D	E
0	1.2	2018-02-26	-2.718282	This	Same
1	1.2	2018-02-26	1.718282	column	Same
2	1.2	2018-02-26	-1.304068	has	Same

```
In [53]: df_demo["F"] = df_demo["C"] - df_demo["A"]
df_demo.head(3)
```

Out[53]:

	A	B	C	D	E	F
0	1.2	2018-02-26	-2.718282	This	Same	-3.918282
1	1.2	2018-02-26	1.718282	column	Same	0.518282
2	1.2	2018-02-26	-1.304068	has	Same	-2.504068



Aside: Plotting without Pandas

Matplotlib 101

- Matplotlib: de-facto standard for plotting in Python
- Main interface: `pyp̸ot` ; provides MATLAB-like interface
- Better: Use object-oriented API with `Figure` and `Axis`
- Great integration into Jupyter Notebooks
- Since v. 3: Only support for Python 3
- → <https://matplotlib.org/>



Aside: Plotting without Pandas

Matplotlib 101

- Matplotlib: de-facto standard for plotting in Python
- Main interface: `pyplot` ; provides MATLAB-like interface
- Better: Use object-oriented API with `Figure` and `Axis`
- Great integration into Jupyter Notebooks
- Since v. 3: Only support for Python 3
- → <https://matplotlib.org/>

```
In [65]: import matplotlib.pyplot as plt  
         %matplotlib inline
```



Task 4

- Sort the Nest data frame by threads
- Plot "Presim. Time / s" and "Sim. Time / s" of our data frame df as a function of threads
- Use a dashed, red line for "Presim. Time / s", a blue line for "Sim. Time / s" (see [API description](#))
- Don't forget to label your axes and to add a legend (*1st rule of plotting*)
- Tell me when you're done with status icon in BigBlueButton: 👍



Task 4

- Sort the Nest data frame by threads
- Plot "Presim. Time / s" and "Sim. Time / s" of our data frame `df` as a function of threads
- Use a dashed, red line for "Presim. Time / s", a blue line for "Sim. Time / s" (see [API description](#))
- Don't forget to label your axes and to add a legend (*1st rule of plotting*)
- Tell me when you're done with status icon in BigBlueButton: 👍

```
In [71]: df.sort_values(["Threads", "Nodes", "Tasks/Node", "Threads/Task"], inplace=True) # multi-level sort
```

```
In [72]: df.columns
```

```
Out[72]: Index(['id', 'Nodes', 'Tasks/Node', 'Threads/Task', 'Runtime Program / s',
              'Scale', 'Plastic', 'Avg. Neuron Build Time / s',
              'Min. Edge Build Time / s', 'Max. Edge Build Time / s',
              'Min. Init. Time / s', 'Max. Init. Time / s', 'Presim. Time / s',
              'Sim. Time / s', 'Virt. Memory (Sum) / kB', 'Local Spike Counter (Sum)',
              'Average Rate (Sum)', 'Number of Neurons', 'Number of Connections',
              'Min. Delay', 'Max. Delay', 'Threads'],
              dtype='object')
```

```
In [73]: fig, ax = plt.subplots()
ax.plot(df["Threads"], df["Presim. Time / s"], linestyle="dashed", color="red", label="Presim. Time / s")
ax.plot(df["Threads"], df["Sim. Time / s"], "-b", label="Sim. Time / s")
ax.set_xlabel("Threads")
ax.set_ylabel("Time / s")
ax.legend(loc='best');
```



Plotting with Pandas

- Each data frame has a `.plot()` function (see [API](#))
- Plots with Matplotlib
- Important API options:
 - `kind`: 'line' (default), 'bar[h]', 'hist', 'box', 'kde', 'scatter', 'hexbin'
 - `subplots`: Make a sub-plot for each column (good together with `sharex`, `sharey`)
 - `figsize`
 - `grid`: Add a grid to plot (use Matplotlib options)
 - `style`: Line style per column (accepts list or dict)
 - `logx`, `logy`, `loglog`: Logarithmic plots
 - `xticks`, `yticks`: Use values for ticks
 - `xlim`, `ylim`: Limits of axes
 - `yerr`, `xerr`: Add uncertainty to data points
 - `stacked`: Stack a bar plot
 - `secondary_y`: Use a secondary `y` axis for this plot
 - Labeling
 - `title`: Add title to plot (Use a list of strings if `subplots=True`)
 - `legend`: Add a legend
 - `table`: If `true`, add table of data under plot
 - `**kwargs`: Non-parsed keyword passed to Matplotlib's plotting methods



TASK

Task 5

Use the Nest data frame `df` to:

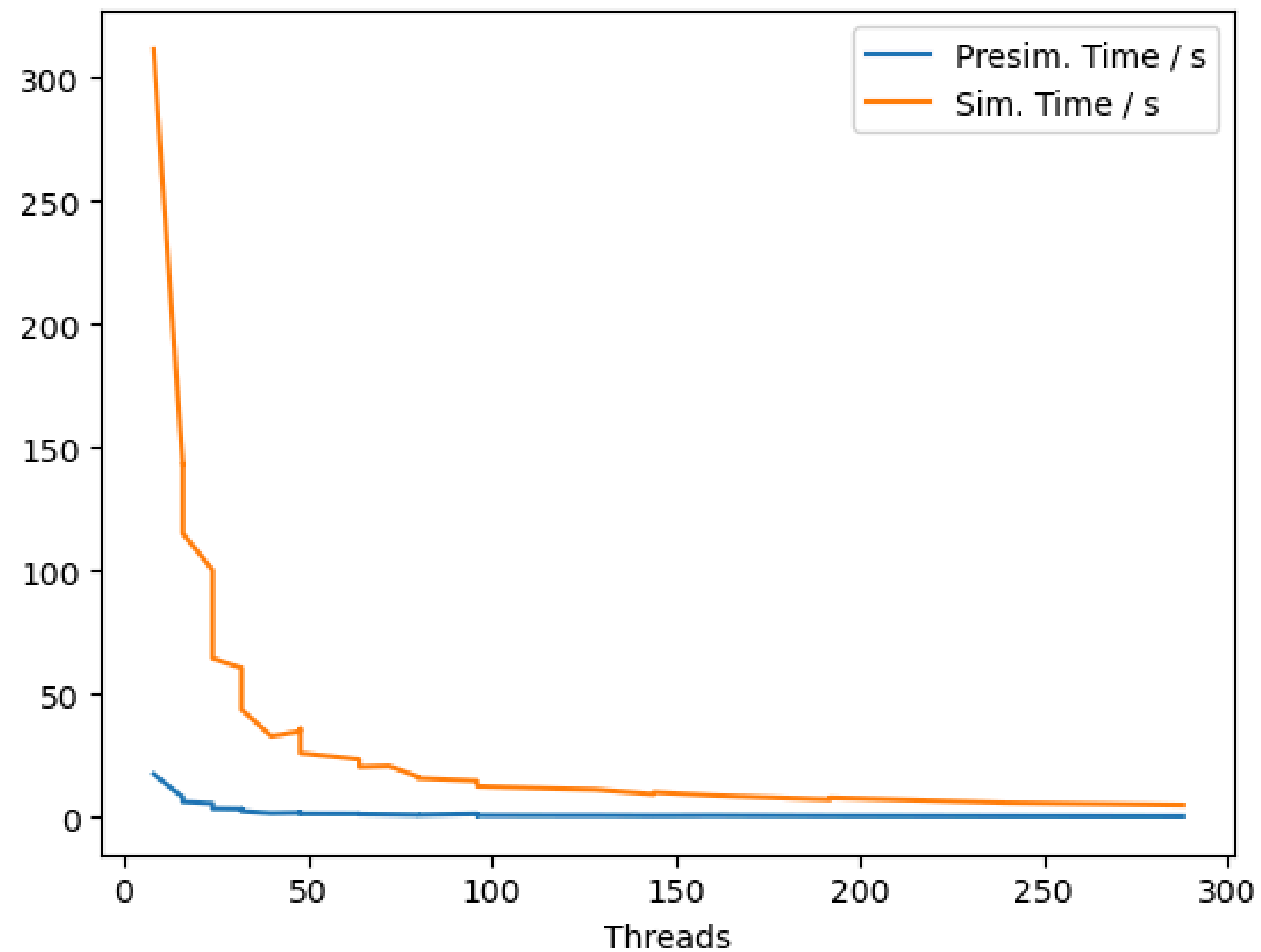
1. Make threads index of the data frame (`.set_index()`)
2. Plot `"Presim. Time / s"` and `"Sim. Time / s"` individually
3. Plot them onto one common canvas!
4. Make them have the same line colors and styles as before
5. Add a legend, add missing axes labels
6. Tell me when you're done with status icon in BigBlueButton: 👍



More Plotting with Pandas

Recap: Our first proper Pandas plot

```
In [84]: df[["Presim. Time / s", "Sim. Time / s"]].plot();
```

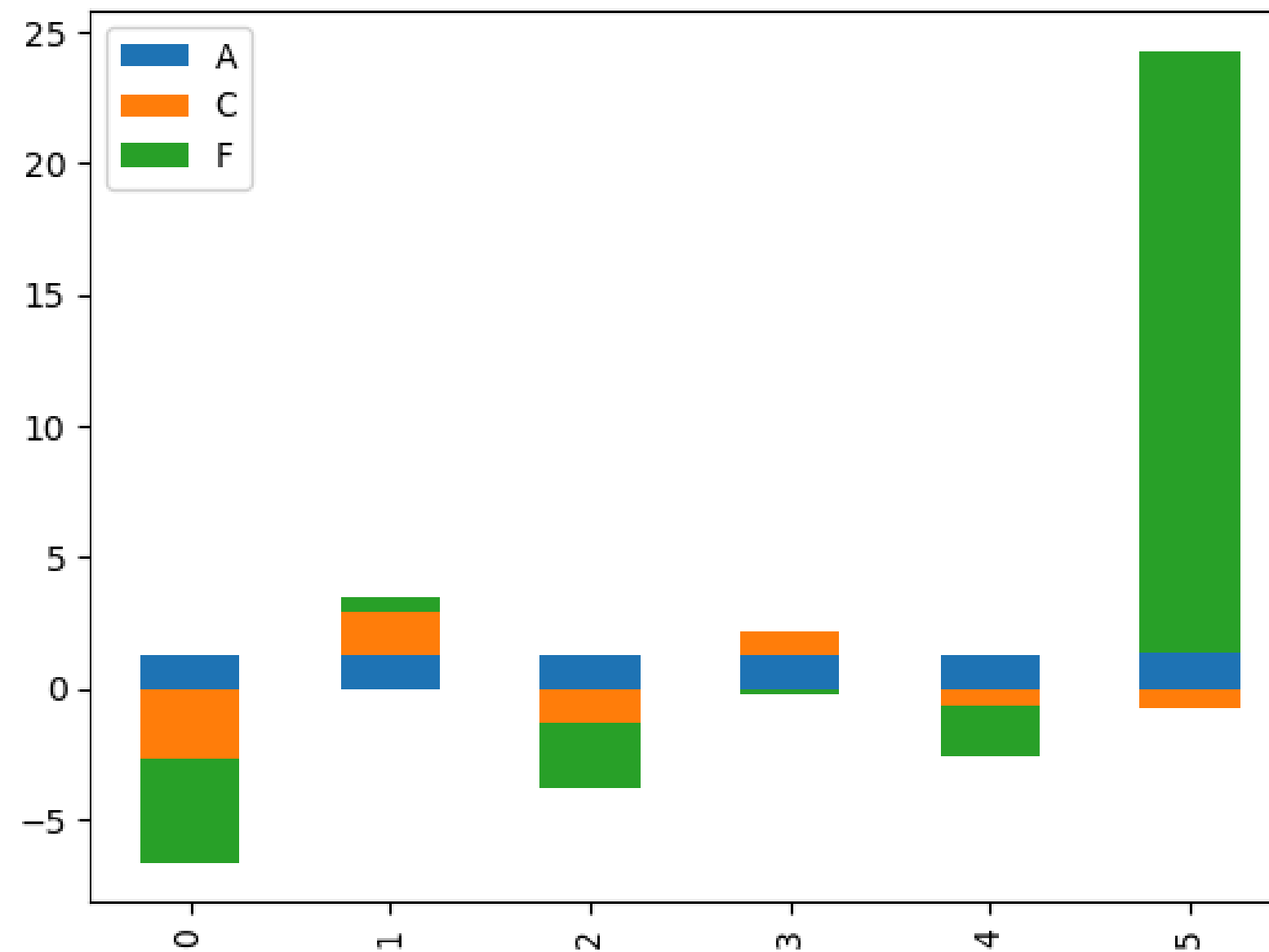


- That's why I think Pandas is great!

More Plotting with Pandas

Some versatility

```
In [85]: df_demo[["A", "C", "F"]].plot(kind="bar", stacked=True);
```



Combine Pandas with Matplotlib

- Pandas shortcuts very handy
- But sometimes, one needs to access underlying Matplotlib functionality
- No problemo!
- **Option 1:** Pandas always returns axis
 - Use this to manipulate the canvas
 - Get underlying `figure` with `ax.get_figure()` (for `fig.savefig()`)
- **Option 2:** Create figure and axes with Matplotlib, use when drawing
 - `.plot()`: Use `ax` option

Feature	Use pandas <code>.plot()</code>	Use matplotlib
Quick plots	✓	✗
Styling (<code>style=</code>)	✓	⚠ manual
Error bars	✗ (limited)	✓
Publication-quality	✗	✓



Aside: Seaborn

- Python package on top of Matplotlib
- Powerful API shortcuts for plotting of statistical data
- Manipulate color palettes
- Works well together with Pandas
- Also: New, well-looking defaults for Matplotlib (IMHO)
- → <https://seaborn.pydata.org/>

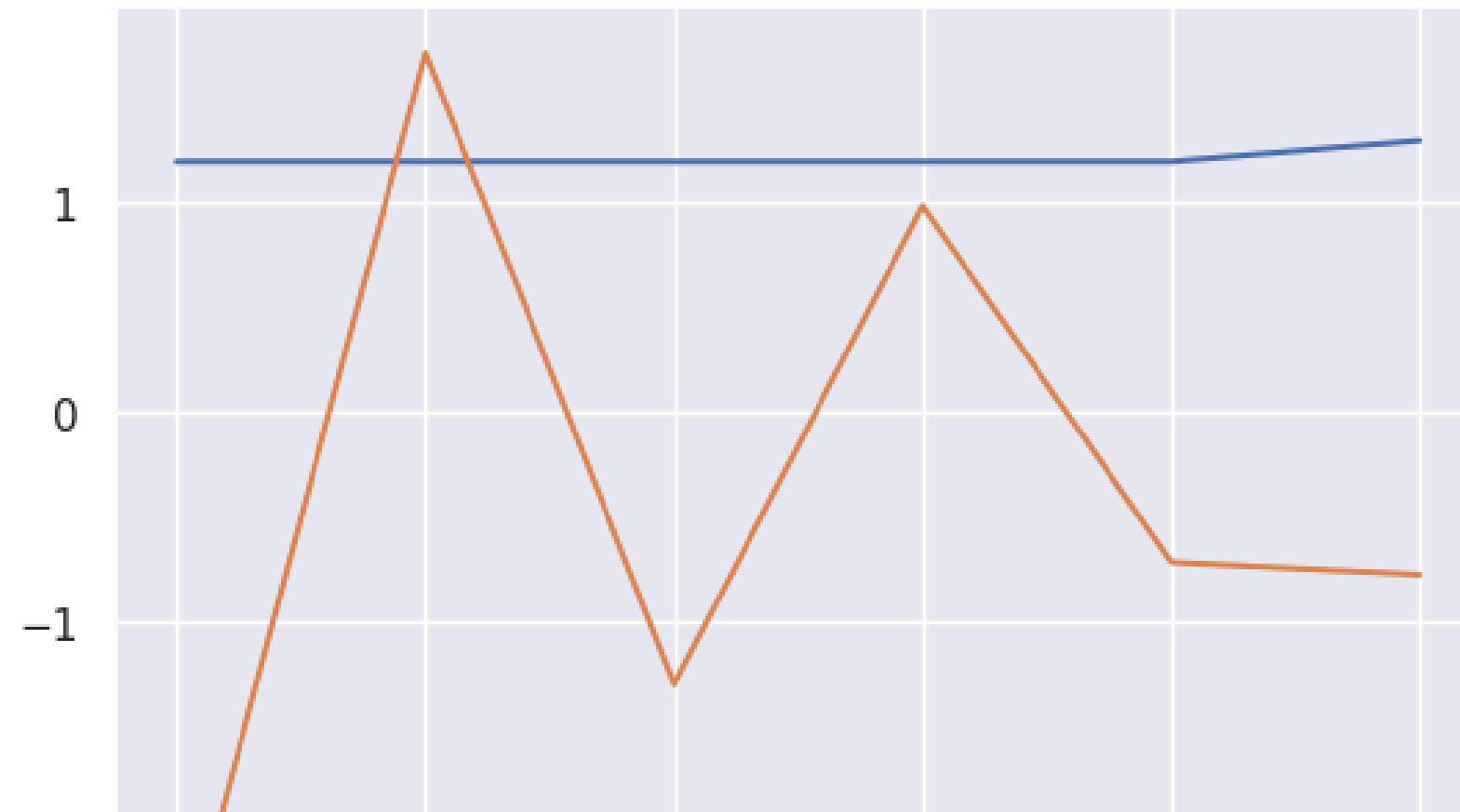


Aside: Seaborn

- Python package on top of Matplotlib
- Powerful API shortcuts for plotting of statistical data
- Manipulate color palettes
- Works well together with Pandas
- Also: New, well-looking defaults for Matplotlib (IMHO)
- → <https://seaborn.pydata.org/>

```
In [93]: import seaborn as sns  
sns.set() # set defaults
```

```
In [94]: df_demo[["A", "C"]].plot();
```



Task 6

- To your `df` Nest data frame, add a column with the unaccounted time (`Unaccounted Time / s`), which is the difference of program runtime, average neuron build time, minimal edge build time, minimal initialization time, presimulation time, and simulation time.

(I know this is technically not super correct, but it will do for our example.)

- Plot a stacked bar plot of all these columns (except for program runtime) over the threads
- Tell me when you're done with status icon in BigBlueButton: 👍



Task 6

- To your `df` Nest data frame, add a column with the unaccounted time (`Unaccounted Time / s`), which is the difference of program runtime, average neuron build time, minimal edge build time, minimal initialization time, presimulation time, and simulation time.
(I know this is technically not super correct, but it will do for our example.)
- Plot a stacked bar plot of all these columns (except for program runtime) over the threads
- Tell me when you're done with status icon in BigBlueButton: 👍

```
In [104]: cols = [  
    'Avg. Neuron Build Time / s',  
    'Min. Edge Build Time / s',  
    'Min. Init. Time / s',  
    'Presim. Time / s',  
    'Sim. Time / s'  
]  
df["Unaccounted Time / s"] = df['Runtime Program / s']  
for entry in cols:  
    df["Unaccounted Time / s"] = df["Unaccounted Time / s"] - df[entry]
```



Next *Level*: Hierarchical Data

- `MultiIndex` only a first level
- More powerful:
 - Grouping: `.groupby()` ("Split-apply-combine", [API](#), [User Guide](#))
 - Pivoting: `.pivot_table()` ([API](#), [User Guide](#)); also `.pivot()` (specialized version of `.pivot_table()`, [API](#))

```
In [109]: df.groupby("Nodes").mean()
```

Out[109]:

	id	Tasks/Node	Threads/Task	Runtime Program / s	Scale	Plastic	Avg. Neuron Build Time / s	Min. Edge Build Time / s	Max. Edge Build Time / s	Min In Time
Nodes										
1	5.333333	3.0	8.0	185.023333	10.0	1.0	0.220000	42.040000	42.838333	0.5833
2	5.333333	3.0	8.0	73.601667	10.0	1.0	0.168333	19.628333	20.313333	0.1916
3	5.333333	3.0	8.0	43.990000	10.0	1.0	0.138333	12.810000	13.305000	0.1350
4	5.333333	3.0	8.0	31.225000	10.0	1.0	0.116667	9.325000	9.740000	0.0883
5	5.333333	3.0	8.0	24.896667	10.0	1.0	0.140000	7.468333	7.790000	0.0700
6	5.333333	3.0	8.0	20.215000	10.0	1.0	0.106667	6.165000	6.406667	0.0516

6 rows × 21 columns



Task 7

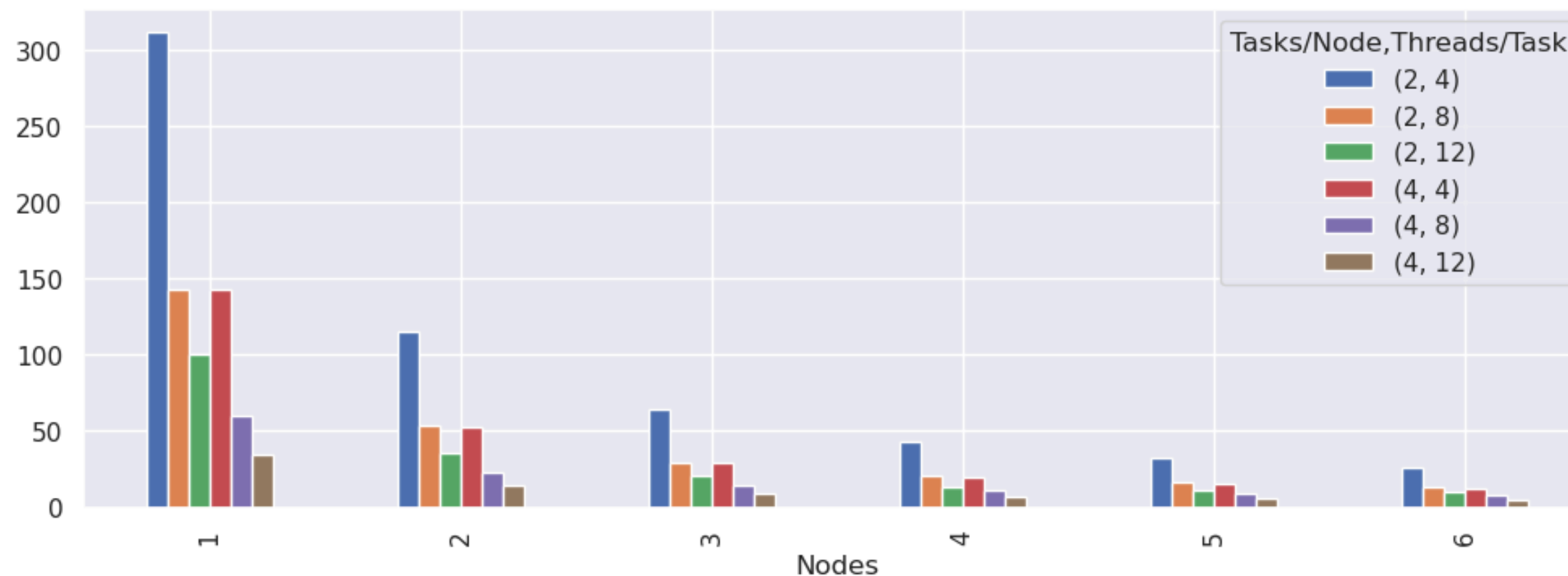
- Create a pivot table based on the Nest `df` data frame
- Let the `x` axis show the number of nodes; display the values of the simulation time `"Sim. Time / s"` for the tasks per node and threads per task configurations
- Please plot a bar plot
- Tell me when you're done with status icon in BigBlueButton: 👍



Task 7

- Create a pivot table based on the Nest `df` data frame
- Let the `x` axis show the number of nodes; display the values of the simulation time `"Sim. Time / s"` for the tasks per node and threads per task configurations
- Please plot a bar plot
- Tell me when you're done with status icon in BigBlueButton: 👍

```
In [117]: df.pivot_table(  
    index="Nodes",  
    columns=["Tasks/Node", "Threads/Task"],  
    values="Sim. Time / s",  
).plot(kind="bar", figsize=(12, 4));
```



Large Data & Mangling

- Pandas can read data directly in `tar` form
- Pandas can read data directly from online resource
- Let's combine that to an advanced task!

Task 8 (*MATSE Bonus*)

- Create bar chart of top 10 actors (on `x`) and average ratings of their top movies (`y`) based on IMDb data (only if they play in at least two movies)



Task 8 (*MATSE Bonus*)

- Create bar chart of top 10 actors (on `x`) and average ratings of their top movies (`y`) based on IMDb data (only if they play in at least two movies)

- IMDb provides data sets at datasets.imdbws.com
- Can directly be loaded like

```
pd.read_table('https://datasets.imdbws.com/dataset.tsv.gz', sep="\t", low_memory=False, na_values=["\N", "nan"])
```

- Needed:
 - `name.basics.tsv.gz` (for names of actors and movies they are known for)
 - `title.ratings.tsv.gz` (for ratings of titles)
- Strategy *suggestions*:
 - Use `df.apply()` with custom function
 - Custom function: Compute average rating and determine if this entry is eligible for plotting (this *can* be done at once, but does not need to be)
 - Average rating: Look up title IDs as listed in `knownForTitles` in titles dataframe



TASK

Task 8B (B, like *Bonus*)

All of the following are unique sub-tasks, which can be done individually

- In addition to Task 8, restrict the top titles to those with more than 10000 votes
- For 30 top-rated actors, plot rating vs. age
- For 30 top-rated actors, plot rating vs. average runtime of the known-for-titles (using `title.basics.tsv.gz`)

Random Features Not Shown

This are all links:

- `df.drop()`
- `df.corr()`
- `df.boxplot()`
- `pd.read_sql_query("SELECT * FROM purchases", con)`
- `df.duplicated()` and `df.drop_duplicates()`
- Aliases for [categorical data](#)
- Working with [time](#)
 - `ts.tz_convert`
 - `pd.period_range()`
 - `pd.period_range().asfreq()`

Conclusion

- Pandas works with and on **data frames**, which are central
- **Slice** frames to your likings
- **Plot** frames
 - Together with Matplotlib, Seaborn, others
- **Pivot** tables are next level greatness
- Remember: ***Pandas as early as possible!***
- Thanks for being here! 🥰

Feedback to a.herten@fz-juelich.de, a.dabah@fz-juelich.de

Next slide: Further reading

Further Reading

- [Pandas User Guide](#)
- [Matplotlib and LaTeX Plots](#)
- [towardsdatascience.com](#):
 - [Pandas DataFrame: A lightweight Intro](#)
 - [Introduction to Data Visualization in Python](#)
 - [Basic Time Series Manipulation with Pandas](#)
 - [Mapping with Matplotlib, Pandas, Geopandas and Basemap in Python](#)

In []: